Java Inner Classes

The Complete Beginner-Friendly Guide

codehive

Introduction

In Java, a class can be declared inside another class. These are called **inner classes**, and they are a powerful feature of the Java language. Their primary goal is to help group related code together for better readability and maintainability.

Why use Inner Classes?

- It groups classes that belong together.
- It increases encapsulation (private data hiding).
- It can lead to more readable and maintainable code.

1. Accessing an Inner Class

To access an inner class, you must first create an object of the outer class, and then create an object of the inner class using a specific syntax.

```
class OuterClass {
  int x = 10;

  class InnerClass {
    int y = 5;
  }
}

public class Main {
  public static void main(String[] args) {
    // Step 1: Instantiate Outer Class
    OuterClass myOuter = new OuterClass();

    // Step 2: Instantiate Inner Class through Outer
    OuterClass.InnerClass myInner = myOuter.new InnerClass();

    System.out.println(myInner.y + myOuter.x);
  }
}
```

Output:

```
15 (5 + 10)
```

2. Private Inner Classes

Unlike regular classes, inner classes can be marked as **private** or **protected**. If you don't want external objects to access the inner class, you should declare it as private.

```
class OuterClass {
  int x = 10;

  private class InnerClass {
    int y = 5;
  }
}

public class Main {
  public static void main(String[] args) {
    OuterClass myOuter = new OuterClass();
    // This will generate an ERROR
    OuterClass.InnerClass myInner = myOuter.new InnerClass();
  }
}
```

Compilation Error:

```
Main.java:13: error: OuterClass.InnerClass has private access
in OuterClass
```

3. Static Inner Class

An inner class can also be **static**. This means you can access it without creating an object of the outer class first.

```
class OuterClass {
  int x = 10;

  static class InnerClass {
    int y = 5;
  }
}

public class Main {
  public static void main(String[] args) {
    // No OuterClass object needed!
    OuterClass.InnerClass myInner = new OuterClass.InnerClass();
    System.out.println(myInner.y);
  }
}
```

Important Note: Just like static methods, a static inner class does not have access to non-static members of the outer class.

4. Accessing Outer Class From Inner Class

One major advantage of inner classes is their ability to access attributes and methods of the outer class directly.

```
class OuterClass {
  int x = 10;

  class InnerClass {
    public int myInnerMethod() {
      // Accessing x from the parent class
      return x;
    }
  }
}

public class Main {
  public static void main(String[] args) {
    OuterClass myOuter = new OuterClass();
    OuterClass.InnerClass myInner = myOuter.new InnerClass();
    System.out.println(myInner.myInnerMethod());
  }
}
```

Output: 10

Summary Checklist

Keep these points in mind when working with Java Inner Classes:

- **Nesting Class:** A class within a class.
- **Access:** Create outer object first, then inner object.
- **Visibility:** Can be private or protected (unlike normal classes).
- **Static:** Static inner classes can be instantiated independently.
- **Context:** Inner classes can "see" everything in the outer class.

End of Lesson - Developed by **codehive**